Laboratory 1

Variant 4
Group 19
By Mathis Morra-Fischer & Nolan Capomaggio

Introduction

The Task:

The objective of this laboratory is to implement the Newton's method to minimise the two variable function below:

$$f(x, y) = 2sin(x) + 3cos(y)$$

This task involves analytically calculating the gradient vector and the precalculated Hessian matrix to guide the numerical search toward a point where the function's slope is zero and its curvature is positive (the local minimum).

The Algorithm:

Newton's method for optimization is a second-order algorithm. It uses a local quadratic approximation of the function to find the stationary points $(\nabla f = 0)$. The update rule is defined as:

$$xk + 1 = xk - [Hf(xk)^{-1}] \nabla f(xk)$$

- **Usage:** High-precision local optimization, training logistic regression, and solving non-linear engineering problems.
- **Advantages:** Quadratic convergence speed (reaches the optimum in very few iterations compared to Gradient Descent).
- **Disadvantages:** It requires the calculation and inversion of the Hessian matrix. It is also sensitive to the starting point; if the initial guess is poor, it may converge to a maximum or a saddle point (which is the case with the initial guess given in the template).

Implementation

The implementation starts from the given function:

```
def function(x, y):
    return 2 * np.sin(x) + 3 * np.cos(y)
```

For this function the gradient is:

$$\nabla f(x, y) = \begin{bmatrix} 2\cos(x) \\ -3\sin(y) \end{bmatrix}$$

And the hessian matrix is:

$$H_f(x, y) = \begin{bmatrix} -2\sin(x) & 0 \\ 0 & -3\cos(y) \end{bmatrix}.$$

These expressions are used directly inside the Newton method.

Newton method:

The program starts from an initial guess and stores the successive visited point to visualize the path of the algorithm.

```
current_guess = np.array(initial_guess, dtype=float)
path = [current_guess.copy()]
```

At each iteration, the gradient is computed:

```
gradient = np.array([
    2 * np.cos(x),
    -3 * np.sin(y)
])
```

Then the norm of the gradient is checked if it's smaller than the tolerance.

```
grad_norm = np.linalg.norm(gradient)
if grad_norm < tol:
    return current_guess.tolist(), i, path
```

Next the hessian is built, the determinant is computed. If it’s too close to zero the Newton step cannot be computed reliably, so the algorithm stops:

```
det_hessian = np.linalg.det(hessian)
if abs(det_hessian) < 1e-10:
    return current_guess.tolist(), i, path
```

If the hessian is valid , Newton direction is computed and the point is updated:

```
newton_direction = -np.linalg.inv(hessian) @ gradient
current_guess = current_guess + alpha * newton_direction
path.append(current_guess.copy())
```

This process is repeated until converge or until the maximum number of iterations is reached.

Visualization:

The visualization function creates a 3D surface of the given function on the domain $[-5, 5] \times [-5, 5]$

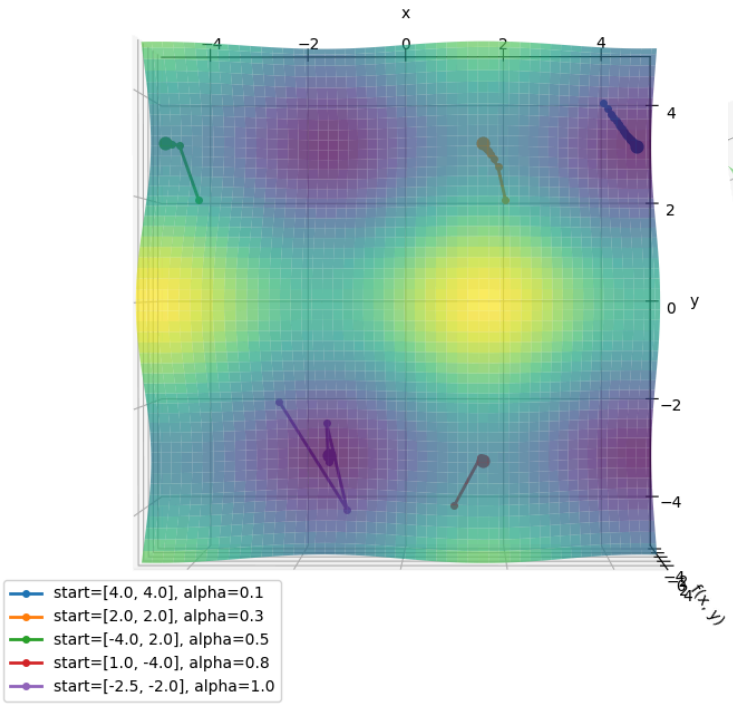
```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = function(X, Y)
```

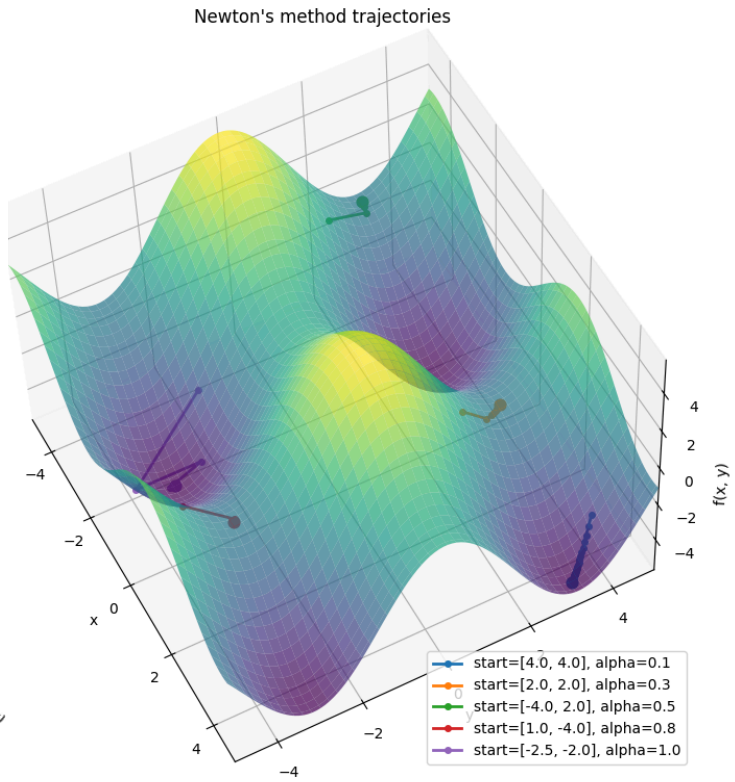
The surface is plotted using plot_surface, and then the trajectories of Newton's method are added for each tested pair (initial_guess, alpha). Each trajectory is displayed as a line with markers corresponding to the successive iterations. The visualization shows the shape of the function, the starting point of each test, the path followed by the algorithm and the final point reached by Newton's method. The test cases used are listed in the example array of initial_guess, alpha pairs:

```
examples = [
    {"initial_guess": [4.0, 4.0], "alpha": 0.1},
    {"initial_guess": [2.0, 2.0], "alpha": 0.3},
    {"initial_guess": [-4.0, 2.0], "alpha": 0.5},
    {"initial_guess": [1.0, -4.0], "alpha": 0.8},
    {"initial_guess": [-2.5, -2.0], "alpha": 1.0},
]
```

Discussion

Results Analysis:





Using an initial point $x0 = [4.0, 4.0]$, the algorithm successfully converged to the point $x \approx [4.712, 3.141] \approx [3\pi/2, \pi]$ in 141 iterations (because alpha is really small, alpha = 0.1). At this point, the function value is $f = -5$, confirming we found a global minimum.

Test Cases

1. **Near-Minimum Start:** Starting at [4.0,4.0], the Hessian is positive definite, leading to rapid convergence to the minimum of z = -5 in 141 iterations.
2. **Near-Maximum Start:** Starting near [2.0,2.0] causes the algorithm to converge to a local minimum because the algorithm only seeks a point where the gradient is zero. but this is only a saddle point.
3. **Near-Border Start:** Starting near the border of the allowed range [-4.0, 2.0] can lead to hitting the border and considering it the local minimum.
4. **High alpha value:** Using a high alpha value (alpha = 1) can lead to the right solution but not converging in a straight line to it.

Conclusion

What we learned: During this lab we understood the importance of finding the best initial guess to avoid being stuck in a saddle point. On the other hand, implementing Newton's method gave us the opportunity to understand the link between the Hessian eigenvalue and the slope rate in order to find the lowest point.

Difficulties: The nature of the function leads to lots of saddle points due to its topography. Selecting a good starting point was a challenge in order to get the real minimum of the function.

Future improvements: We could implement another function which would explore different initial guesses and returning the lowest value to avoid giving saddle points as a result.